Next.js — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Setup

02 App Router and File-based Routing

03 Server and Client Components

04 Links, Navigation and Layouts

05 Styling

06 Images and Metadata

Intermediate (6 topics)

07 Data Fetching in Server Components

08 Dynamic Routes and generateStaticParams

09 Route Handlers (API)

10 Loading, Error and Not-found UI

11 Forms and Server Actions

12 Environment Variables and Config

Advanced (6 topics)

13 Rendering Strategies: SSG, SSR, ISR

14 Caching and Revalidation

15 Authentication

16 Middleware

17 Database Integration

18 Streaming and Suspense

Master (4 topics)

19	Performance Optimisation
20	Testing and CI
21	Real Project: Full-stack Blog or Store
22	Deployment and Self-hosting

Beginner Level

01. Introduction and Setup

Next.js is a React framework with routing, server rendering, API routes and optimisation built in.

```
npx create-next-app@latest my-app
cd my-app && npm run dev
```

02. App Router and File-based Routing

Folders inside `app/` become URLs; `page.tsx` is the page, `layout.tsx` wraps pages.

```
app/
  layout.tsx
  page.tsx           -> /
  about/page.tsx     -> /about
  blog/[slug]/page.tsx -> /blog/hello
```

03. Server and Client Components

Components are **Server Components** by default (no client JS). Add `'use client'` for state, effects and event handlers.

```
'use client';
import { useState } from 'react';
export default function Counter() { const [n, setN] = useState(0); return <button onClick={()
=> setN(n + 1)}>{n}</button>; }
```

04. Links, Navigation and Layouts

Use `<Link>` for client-side navigation and nested layouts for shared UI; `useRouter` for programmatic navigation.

```
import Link from 'next/link';
<Link href="/about">About</Link>
```

05. Styling

CSS Modules, global CSS, Tailwind CSS and CSS-in-JS options; fonts via `next/font`.

```
import styles from './page.module.css';
<h1 className={styles.title}>Hi</h1>
```

06. Images and Metadata

`next/image` serves optimised images; export `metadata` for SEO titles and descriptions.

```
export const metadata = { title: 'My Site', description: 'Learn Next.js' };
```

Intermediate Level

07. Data Fetching in Server Components

Server Components can be `async` and fetch data directly; control caching with `fetch` options and `revalidate`.

```
export default async function Page() {
  const res = await fetch('https://api.example.com/posts', { next: { revalidate: 60 } });
  const posts = await res.json();
  return <ul>{posts.map((p: any) => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

08. Dynamic Routes and generateStaticParams

Dynamic segments `[id]` receive params; `generateStaticParams` pre-renders pages at build time.

```
export async function generateStaticParams() { return [{ slug: 'a' }, { slug: 'b' }]; }
```

09. Route Handlers (API)

Create API endpoints with `route.ts` exporting HTTP-method functions.

```
// app/api/hello/route.ts
export async function GET() { return Response.json({ message: 'Hello' }); }
```

10. Loading, Error and Not-found UI

Special files `loading.tsx`, `error.tsx`, `not-found.tsx` provide streaming fallbacks and error boundaries per route.

```
// app/blog/loading.tsx
export default function Loading() { return <p>Loading posts...</p>; }
```

11. Forms and Server Actions

Server Actions are async functions marked `'use server'` that run on the server and can be used as form actions.

```
async function create(formData: FormData) {
  'use server';
  await db.post.create({ data: { title: formData.get('title') as string } });
}
<form action={create}>...</form>
```

12. Environment Variables and Config

`.env.local` for secrets; only `NEXT_PUBLIC_` variables reach the browser. Configure in `next.config.ts`.

```
DATABASE_URL=...
NEXT_PUBLIC_API=https://api.example.com
```

Advanced Level

13. Rendering Strategies: SSG, SSR, ISR

Static generation (build), server rendering (each request), incremental regeneration (`revalidate`), and Partial Prerendering; pick per page.

```
export const dynamic = 'force-dynamic'; // SSR
export const revalidate = 3600;          // ISR
```

14. Caching and Revalidation

Understand request memoisation, Data Cache and Router Cache; use `revalidatePath` / `revalidateTag` after mutations.

```
import { revalidatePath } from 'next/cache';
revalidatePath('/blog');
```

15. Authentication

Use Auth.js (NextAuth), sessions in cookies, protect routes with middleware and verify on the server.

```
export { auth as middleware } from '@/auth';
export const config = { matcher: ['/dashboard/:path*'] };
```

16. Middleware

Code that runs before a request completes: redirects, rewrites, auth checks, A/B tests, headers.

```
export function middleware(req: NextRequest) {
  if (!req.cookies.get('token')) return NextResponse.redirect(new URL('/login', req.url));
}
```

17. Database Integration

Use Prisma or Drizzle with PostgreSQL/MySQL inside server components, actions and route handlers.

```
const users = await prisma.user.findMany({ where: { active: true } });
```

18. Streaming and Suspense

Wrap slow parts in `<Suspense>` so the rest of the page streams to the browser immediately.

```
<Suspense fallback={<Skeleton />}><Reviews /></Suspense>
```

Master Level

19. Performance Optimisation

Optimise images/fonts/scripts, reduce client JS with Server Components, analyse bundles, use Core Web Vitals and caching.

```
npm i @next/bundle-analyzer
ANALYZE=true npm run build
```

20. Testing and CI

Unit tests with Vitest/Jest, component tests with Testing Library, end-to-end with Playwright, GitHub Actions pipeline.

```
npx playwright test
```

21. Real Project: Full-stack Blog or Store

Auth, database, CRUD with Server Actions, image upload, SEO metadata, ISR and deployment on Vercel.

22. Deployment and Self-hosting

Deploy to Vercel, or self-host with `next start` / Docker `output: 'standalone'`; configure env vars and caching.

```
npm run build && npm start
```

Interview Question Bank

Q1. Server vs Client Components?
Server Components render on the server and ship no JS; Client Components (`'use client'`) hydrate in the browser and use state/effects.

Q2. SSR vs SSG vs ISR?
SSR renders per request; SSG renders at build; ISR regenerates static pages in the background after a time or event.

Q3. What is the App Router?
The `app/` directory routing system with layouts, server components, streaming and server actions.

Q4. What are Server Actions?
Server-side functions invoked from forms or client code without writing an API route.

Q5. What is middleware used for?
Running logic before requests: auth redirects, rewrites, localisation, headers.

Q6. How does `next/image` help?
Automatic resizing, lazy loading, modern formats and layout shift prevention.

Q7. How do you handle SEO?
`metadata` / `generateMetadata`, server rendering, sitemaps, canonical URLs and structured data.

Q8. What is hydration?
React attaching event handlers to server-rendered HTML in the browser.

Q9. How are environment variables exposed?
Only variables prefixed `NEXT_PUBLIC_` are bundled for the browser; others stay server-side.

Q10. Pages Router vs App Router?
Pages Router uses `pages/` with `getServerSideProps`; App Router is the newer, component-based model.

Capstone Project Ideas

1. Personal blog with MDX
2. E-commerce store with Stripe
3. Dashboard with auth and Postgres
4. URL shortener
5. Job board with search